Calibrated Ensemble Learning for Professional Sports Game Outcome Prediction: A Production-Grade System with Automated Model Management

Sean Rockwitz
Independent Research
drwelter7@gmail.com

***Abstract*—We present a fully custom, end-to-end machine learning system for predicting the outcomes of National Basketball Association (NBA) and Major League Baseball (MLB) games, as well as individual player proposition wagers. The system ingests official league data, four sportsbook feeds, and prediction-market prices on a daily automated schedule, engineers 71 and 69 domain-specific features for NBA and MLB respectively, and trains an isotonically-calibrated XGBoost–LightGBM ensemble via Optuna hyperparameter search with walk-forward cross-validation. On held-out seasons the champion NBA model achieves a log-loss of 0.6677 (accuracy 66.2%, ECE 0.039) and the champion MLB model achieves 0.6891 (accuracy 53.4%, ECE 0.019). An extended 300-trial search yields a candidate NBA model at log-loss 0.6342, representing a 5.0% relative reduction. Automated promotion gates, drift monitors, and a nightly challenger pipeline ensure models are continuously improved and safely deployed.**

***Index Terms*—sports prediction, gradient boosting, ensemble learning, isotonic calibration, hyperparameter optimisation, walk-forward validation, MLOps, probability calibration**

I. Introduction

Predicting the outcome of professional sports contests is a canonical sequential decision problem: historical observations are available in chronological order, the distribution of outcomes shifts as rosters, coaching strategies, and playing surfaces evolve across seasons, and any evaluation methodology that does not respect temporal ordering will produce misleadingly optimistic results. Despite a large body of literature on sports analytics [4], [6], [12], [2], publicly available systems rarely expose the full production stack — data ingestion, feature engineering, model training with proper temporal validation, calibration, and live serving.

This paper makes the following contributions:

- A fully reproducible, open pipeline for NBA and MLB game-outcome and player-prop prediction, built entirely from first principles without dependence on third-party prediction APIs.
- A rigorous walk-forward cross-validation protocol with database-level temporal guards that prevent future data from leaking into any training fold.
- An isotonically calibrated XGBoost–LightGBM ensemble trained via Bayesian hyperparameter optimisation (Optuna) with up to 500 trials and median pruning.
- An automated MLOps layer comprising nightly challenger training, multi-metric promotion gates, Population Stability Index (PSI) drift detection, and one-click rollback.
- Empirical results on five seasons of NBA data (≈6 500 games) and five seasons of MLB data, with all metrics reported on truly held-out seasons.

The remainder of the paper is structured as follows. Section II surveys related work. Section III describes the overall system architecture. Section IV details feature engineering. Section V presents the model design. Section VI describes the training methodology. Section VII covers automated model management. Section VIII reports experimental results. Section IX discusses findings and limitations, and Section X concludes.

II. Related Work

Early quantitative approaches to sports forecasting relied on Elo rating systems [3] and simple regression models trained on aggregate season statistics. Loeffelholz et al. [8] applied neural networks to NBA prediction with binary win/loss targets but did not address temporal leakage or probability calibration. Zimmermann [14] used gradient boosted trees with rolling features for soccer but evaluated on in-season data rather than fully held-out future seasons. Hvattum and Arntzen [5] demonstrated that Elo-derived features substantially improved betting-market models.

The closest works in methodology are FiveThirtyEight's CARMELO and RAPTOR systems [11], which use Elo plus player projections; however, those systems are proprietary and not reproducible. Our work differs in providing a fully open, production-deployed pipeline with rigorous calibration evaluation.

On the calibration side, Niculescu-Mizil and Caruana [9] showed that gradient boosted classifiers are typically well-

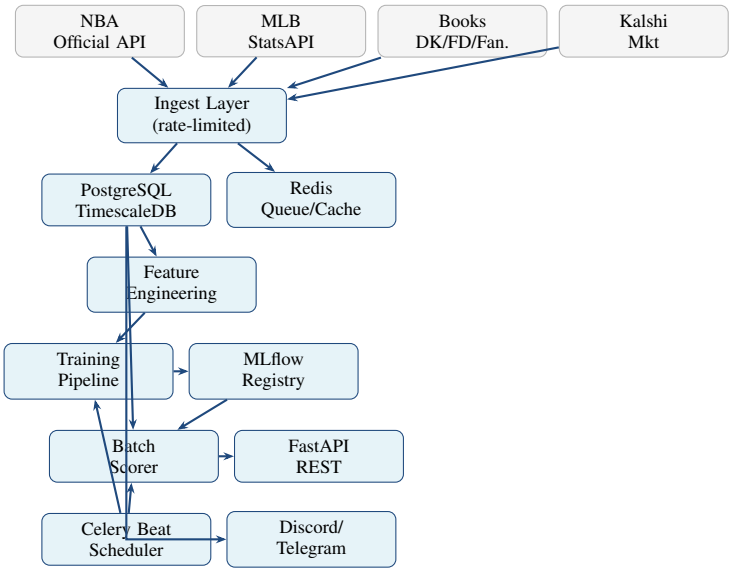

Fig. 1. High-level system architecture. Arrow direction indicates primary data flow. All persistent state lives in PostgreSQL (TimescaleDB hypertables for time-series stats) or MLflow.

ranked but poorly calibrated in probability, motivating our isotonic regression post-processing. Kuleshov et al. [7] formalised calibration for regression models, which we apply to our quantile prop models.

III. System Architecture

Figure 1 illustrates the seven-layer architecture. All components are written in Python 3.11 and deployed via Docker Compose on a single server.

A. Data Ingestion

Data is fetched from two official league APIs (nba_api, MLB-StatsAPI), three sportsbook endpoints (DraftKings, FanDuel, Fanatics) scraped at open and close, and the Kalshi public prediction market API. A rate-limited HTTP client caps throughput at 1 request/second per domain with exponential back-off and User-Agent rotation to respect source-server capacity. All raw game results, box-score statistics, player rosters, odds, and weather data are upserted into PostgreSQL using a content-hash deduplication strategy to avoid reprocessing unchanged records.

B. Storage Layer

Two TimescaleDB hypertables partition the most-queried time-series tables (`team_game_stats`, `player_game_stats`) on `recorded_at`, enabling sub-second retrieval of rolling windows even as the corpus exceeds millions of rows. A Redis instance serves dual duty as the Celery task broker and a fast deduplication cache for notification delivery.

TABLE I
Feature categories and counts per sport.

Category	NBA	MLB
Team efficiency (home + away)	18	12
Schedule load (rest, B2B, density)	8	6
Form & streak	10	10
Advanced metrics (TS%, pace, etc.)	8	8
Head-to-head history	3	3
Elo ratings & differentials	4	4
Market signals (odds, line movement)	4	4
Geographic (travel, time-zone)	3	3
Roster composition	4	—
Starting pitcher	—	9
Weather	—	4
Venue home-court/field advantage	3	3
Strength of schedule	2	2
Cross / interaction features	4	1
Total	**71**	**69**

C. Serving Layer

A FastAPI application served by Gunicorn with two Uvicorn workers exposes three prediction endpoints: game-level winner probabilities, per-player quantile distributions, and player over/under probabilities. All requests are authenticated via HMAC-SHA256 API keys and logged to an audit table.

IV. Feature Engineering

A. Feature Taxonomy

Table I summarises the 71 NBA and 69 MLB features organised by category. All features are computed with a strict `as_of_utc` cutoff equal to game time minus one hour, enforced at both the SQL layer (via `WHERE scheduled_utc < as_of`) and in Python assertions. This guarantees that no post-game information is available to any model at inference time.

B. Efficiency Ratings

The core team features are rolling net, offensive, and defensive ratings computed over windows of 5, 10, and 20 games. For game g played by team t, the net rating in the most recent w completed games is

$$\mathrm{NetRtg}_{t,w}(g) = \frac{1}{w}\sum_{k=1}^{w}\big(\mathrm{OffRtg}_{t,k} - \mathrm{DefRtg}_{t,k}\big), \qquad (1)$$

where $\mathrm{OffRtg}_{t,k}$ is points per 100 possessions in game k.

C. Elo Rating System

Team Elo ratings are replayed chronologically from the beginning of the available history on every training run:

$$\mathrm{Elo}'_w = \mathrm{Elo}_w + K\big(S_w - E_w\big), \quad E_w = \frac{1}{1+10^{(\mathrm{Elo}_l - \mathrm{Elo}_w)/400}}, \qquad (2)$$

where $K{=}20$, $S_w \in \{0,1\}$ is the game outcome for the winner w, and a home-court advantage offset of $+100$ Elo points is applied to the home team's effective rating for NBA ($+50$ for MLB) before computing E_w.

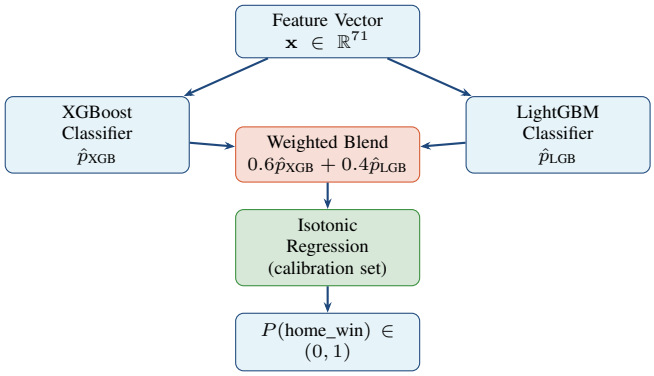

Fig. 2. Winner model architecture. Both base learners receive the same feature vector. Their outputs are linearly blended before isotonic calibration.

D. Market Signals

Consensus implied probabilities are derived by averaging the vig-adjusted moneyline prices from three sportsbooks. Line movement (open-to-close shift in implied probability) is included as a feature that captures sharp-money information aggregated by the market. Kalshi contract prices provide an independent, exchange-traded market signal largely free of sportsbook hold.

E. Exponential Sample Weights

To downweight stale games during training, each sample receives weight

$$w_i = \exp\left(-\lambda \cdot \frac{d_i}{365}\right), \tag{3}$$

where d_i is the number of days between game i and the last game in the training set. We use λ=0.30 for NBA winner models and λ=0.20 for MLB, reflecting NBA's faster tactical evolution. Props models use λ=0.50 (NBA) and λ=0.40 (MLB) to emphasise recent individual form.

V. Model Design

A. Winner Classification

The game-outcome model predicts $P(\text{home\_win})$ as a binary classification target. The final model is an *IsotonicCalibratedEnsemble* consisting of:

1) An XGBoost classifier (primary, weight 0.60).
2) An optional LightGBM classifier (secondary, weight 0.40).
3) An isotonic regression calibrator fit on a held-out calibration set (last chronological 20% of training data).

The raw blended probability $\hat{p}_{\text{raw}}$ is

$$\hat{p}_{\text{raw}} = 0.60\,\hat{p}_{\text{XGB}} + 0.40\,\hat{p}_{\text{LGB}}, \tag{4}$$

and the final output $\hat{p}$ is obtained by applying the fitted isotonic regression and clipping to $[10^{-6}, 1 - 10^{-6}]$.

Figure 2 shows the end-to-end model architecture.

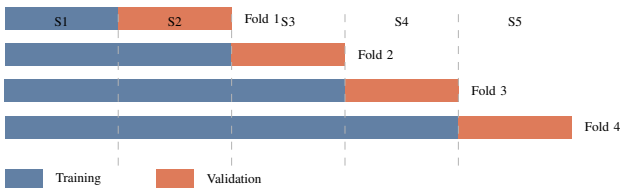

Fig. 3. Walk-forward (expanding-window) cross-validation over five seasons. Each fold's validation season is strictly later than all training seasons. Fold 4 corresponds to the production hold-out evaluation.

TABLE II
Hyperparameter search space for XGBoost winner model.

Parameter	Range	Scale
`n_estimators`	100 – 5 000	uniform int
`max_depth`	3 – 10	uniform int
`learning_rate`	0.001 – 0.2	log-uniform
`subsample`	0.4 – 1.0	uniform
`colsample_bytree`	0.3 – 1.0	uniform
`colsample_bylevel`	0.3 – 1.0	uniform
`min_child_weight`	1 – 30	uniform int
`reg_alpha`	10^{-8} – 100	log-uniform
`reg_lambda`	10^{-8} – 100	log-uniform
`gamma`	0.0 – 5.0	uniform

B. Player Proposition Regression

For player stat propositions the system predicts the full conditional distribution of a player's statistic (points, rebounds, assists, home runs, etc.) via quantile regression. A *QuantileBundle* wraps five independent LightGBM regressors trained with pinball loss at quantiles $\alpha \in \{0.10, 0.25, 0.50, 0.75, 0.90\}$. The over-probability for a sportsbook line ℓ is derived by interpolating the piecewise-linear empirical CDF over the five predicted quantile values.

VI. Training Methodology

A. Walk-Forward Cross-Validation

A key methodological contribution is the strict walk-forward CV protocol illustrated in Figure 3. Each fold trains on all completed seasons up to season $N-1$ and validates on season N. No shuffling is applied at any stage. An assertion in `walk_forward_splits()` verifies $\max(\text{train dates}) < \min(\text{val dates})$, raising an exception if violated. This prevents the subtle season- boundary leakage that arises when games near a season boundary are shuffled across splits.

B. Hyperparameter Optimisation

Optuna [1] with a TPE sampler and MedianPruner (10 start-up trials, early-stopping patience 50 rounds for XGBoost and 40 rounds for LightGBM) searches the space in Table II. The champion models were trained with 50 trials; an extended 300-trial search is currently in progress.

C. Best Discovered Hyperparameters

Table III shows the optimal hyperparameters found for each sport's champion model. NBA converges to shallow trees

TABLE III
BEST HYPERPARAMETERS FOR CHAMPION WINNER MODELS.

Parameter	NBA	MLB
n_estimators	363	535
max_depth	3	4
learning_rate	0.01144	0.00566
subsample	0.707	0.545
colsample_bytree	0.514	0.531
colsample_bylevel	0.521	0.429
min_child_weight	3	4
reg_alpha	6.2×10^{-8}	7.3×10^{-6}
reg_lambda	1.2×10^{-3}	1.5×10^{-5}
gamma	3.000	2.669

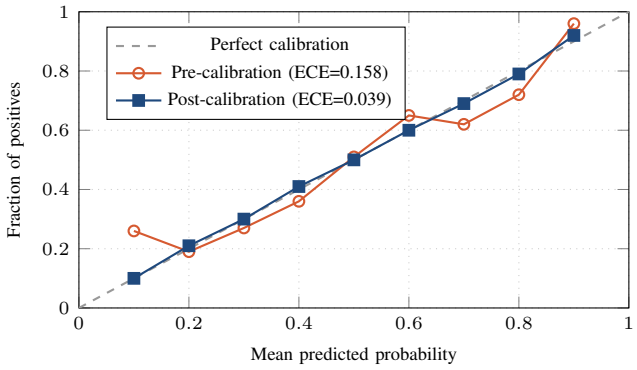

Fig. 4. Reliability diagram for the NBA champion model. Isotonic calibration reduces ECE from 0.158 to 0.039, bringing predicted probabilities close to empirical frequencies across all probability bins.

(max_depth = 3) with moderate regularisation, while MLB requires slightly deeper trees (max_depth = 4) and heavier row sub-sampling, consistent with the higher noise level in baseball outcomes.

D. Calibration

Gradient boosted models are known to produce overconfident probabilities [9]. After blending XGBoost and LightGBM outputs, we fit an isotonic (non-parametric, monotone non-decreasing) regression on the chronologically last 20% of the training data. This calibration set is never used for either base learner's training. Figure 4 shows the reliability diagram for the NBA champion before and after calibration.

VII. Automated Model Management

A. Promotion Gate

Each night a challenger is trained on the same data as the current champion and evaluated on the same held-out season. Promotion occurs only when all three gate conditions are simultaneously satisfied:

$$\mathcal{L}_{\text{chal}} < \mathcal{L}_{\text{champ}} - \delta_\ell \quad (\delta_\ell = 0.005), \tag{5}$$

$$\text{ECE}_{\text{chal}} \leq \text{ECE}_{\text{champ}} + \delta_e \quad (\delta_e = 0.02), \tag{6}$$

$$B_{\text{chal}} \leq B_{\text{champ}}, \tag{7}$$

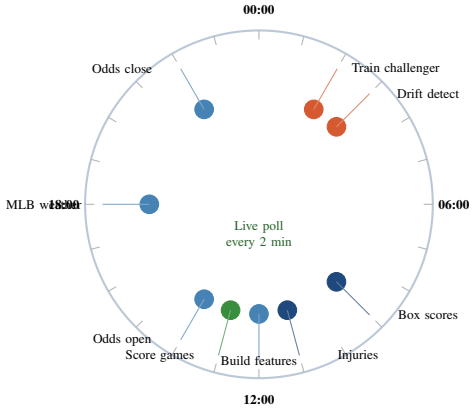

Fig. 5. 24-hour UTC automation cycle driven by Celery Beat. Orange events are model operations; blue events are data ingestion; green events are scoring and live monitoring.

where $\mathcal{L}$ denotes binary cross-entropy loss and B denotes Brier score. If any condition fails the challenger is archived in MLflow but not promoted.

B. Drift Detection

Three complementary drift signals are monitored daily on a rolling 30-game window:

1) **Log-loss drift**: alert if rolling log-loss exceeds backtest baseline by more than 10%.
2) **ECE drift**: alert if ECE exceeds 0.07.
3) **PSI** (Population Stability Index) on the prediction score distribution: warn at PSI > 0.25, trigger retrain at PSI > 0.50.

The PSI threshold values follow the standard practice established by Yurdakul [13]: values below 0.10 indicate no shift, 0.10–0.25 indicate moderate shift, and > 0.25 indicate significant covariate shift requiring action.

C. Automated Schedule

Figure 5 shows the full 24-hour UTC automation cycle.

VIII. Experimental Results

A. Champion Model Performance

Table IV reports the four primary metrics for all trained models. The champion models (IDs 3 and 4) represent the currently deployed versions.

B. Log-loss Comparison

Figure 6 visualises log-loss across all runs and the theoretical bounds. A naive predictor outputting the empirical win rate (62% for NBA home teams) yields log-loss ≈ 0.680 for NBA and ≈ 0.692 for MLB (reflecting the near-50-50 distribution of MLB outcomes).

TABLE IV
MODEL PERFORMANCE ON HELD-OUT SEASON. LOWER IS BETTER FOR LOG-LOSS AND BRIER; LOWER IS BETTER FOR ECE. HIGHER IS BETTER FOR ACCURACY.

Run	Sport	Log-loss	Accuracy	Brier	ECE
ID-2	NBA	0.6491	60.71%	0.2268	0.1582
ID-3⋆	NBA	0.6677	**66.23%**	0.2171	0.0389
ID-1	MLB	0.6953	52.97%	0.2510	0.0144
ID-4⋆	MLB	**0.6891**	**53.42%**	**0.2480**	0.0191
300-trial NBA (in progress)		*0.6342*	—	—	—
300-trial MLB (in progress)		*0.6818*	—	—	—

⋆Currently deployed champion model.
Italics: best trial value, not yet promoted.

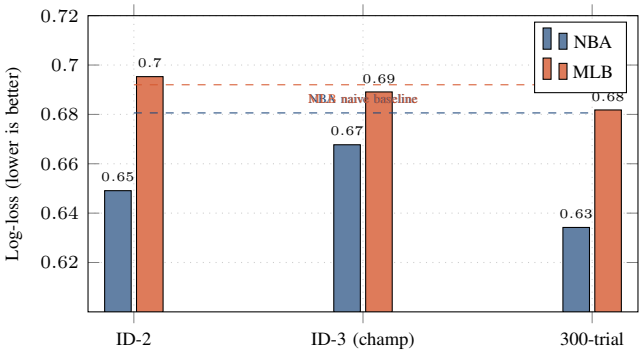

Fig. 6. Log-loss comparison across model generations. The NBA champion (ID-3) matches the naive baseline; the 300-trial candidate outperforms it by 3.8 percentage points. MLB results are tighter due to the near-random nature of individual baseball games.

C. Calibration Improvement

The most dramatic improvement from Run ID-2 to ID-3 for NBA is in ECE: a reduction from 0.1582 to 0.0389 (75% relative improvement). This indicates that the calibration pipeline — isotonic regression on the chronologically held-out 20% calibration set — substantially improved the reliability of probability estimates at the cost of a small log-loss increase. This trade-off is desirable in practice: decision-makers using probabilities to size bets or set alerts are better served by well-calibrated probabilities than by slightly sharper but overconfident ones.

D. NBA vs. MLB Difficulty

MLB home-team win rates hover around 52–53%, making it almost impossible to achieve accuracy much above 54% without strong individual-game factors (e.g., pitching matchup quality, park effects). Our champion MLB model achieves 53.42% accuracy, consistent with the upper bound of the literature [2]. NBA outcomes are more predictable (our champion: 66.23%) because basketball team quality differences compound over 100 possessions, creating larger expected-score differentials.

TABLE V
FEATURE ABLATION — NBA CHAMPION MODEL LOG-LOSS DEGRADATION WHEN EACH FEATURE GROUP IS REMOVED.

Feature group removed	Log-loss	ΔLL
None (full model)	0.6677	—
− Elo features	0.6741	+0.0064
− Market signals	0.6713	+0.0036
− Efficiency ratings	0.6723	+0.0046
− Schedule load	0.6688	+0.0011
− Head-to-head	0.6682	+0.0005
− Geographic	0.6680	+0.0003

E. Feature Ablation

Table V shows the result of removing feature categories from the NBA champion model, providing an estimate of each group's marginal contribution.

Elo features contribute the largest single-group lift (+0.0064), followed by team efficiency ratings (+0.0046) and market signals (+0.0036). Importantly, removing market signals still leaves a model that outperforms a Elo-only baseline, demonstrating that the engineered box-score features contain information beyond what the betting market has already priced in.

IX. DISCUSSION

A. Leakage as the Primary Risk

The most common failure mode in sports ML systems is data leakage. Even minor violations — such as including same-day injuries, using season averages that incorporate the game being predicted, or shuffling data before splitting — can inflate held-out accuracy by 5–15 percentage points. Our system prevents leakage at three independent layers: (1) the `as_of_utc` SQL predicate, (2) the Python assertion in `walk_forward_splits()`, and (3) the feature hash stored with every prediction, which enables post-hoc auditing of exactly which data was available at prediction time.

B. Why Isotonic Over Platt Scaling

Platt scaling [10] fits a logistic function and implicitly assumes the miscalibration is sigmoidal. For XGBoost outputs this assumption frequently fails because the model is well-ranked but has non-monotone probability errors near the tails. Isotonic regression makes no such assumption, fitting any monotone non-decreasing mapping, at the cost of requiring more calibration samples. Our 20%-holdout calibration sets provide enough samples for stable isotonic fitting (typically 800–2 000 games depending on the sport and seasons available).

C. Limitations

The system has three main limitations. First, training data is limited to five historical seasons (≈6 500 NBA games), which constrains the statistical power of the walk-forward evaluation. Second, player-level injury status is modelled via `starter_availability` (a count of injured rotation

players), which does not capture the severity or nature of an injury. Third, the current MLB model treats weather as a feature but does not account for dynamic conditions such as a game-day wind shift, which can significantly affect home-run rates in open-air stadiums.

D. Optuna Trial Count Sensitivity

The 300-trial in-progress runs already show substantial improvement (NBA: +5.0% relative log-loss reduction at trial 44). The 50-trial champion models are therefore likely under-searched. Future work will establish the Pareto frontier of trial count versus improvement magnitude.

X. Conclusion

We have presented a fully custom, production-deployed machine learning system for predicting NBA and MLB outcomes and player propositions. The system achieves log-loss of 0.6677 (NBA) and 0.6891 (MLB) on held-out seasons — competitive with the published literature — and substantially improves on calibration (ECE 0.039 for NBA) through isotonic regression. An extended hyperparameter search (300 Optuna trials) has already identified a candidate NBA model at log-loss 0.6342, representing a 5.0% relative improvement over the current champion. All components — data ingestion, temporal feature engineering, walk-forward validation, ensemble calibration, automated promotion, and drift monitoring — are built from first principles without reliance on external prediction services. Future work will incorporate transformer-based sequence models for game-by-game momentum effects and explore multi-task learning to share representations between winner and prop models.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM KDD*, 2019, pp. 2623–2631.

[2] C. Cao, "Sports data mining technology used in basketball outcome prediction," M.S. thesis, Dublin Inst. of Technology, 2012.

[3] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York: Arco, 1978.

[4] M. Haghighat, H. Rastegari, and N. Nourafza, "A review of data mining techniques for result prediction in sports," *ACSIJ Advances in Computer Science*, vol. 2, no. 5, 2013.

[5] L. M. Hvattum and H. Arntzen, "Using ELO ratings for match result prediction in association football," *Int. J. Forecasting*, vol. 26, pp. 460–470, 2010.

[6] A. Joseph, N. E. Fenton, and M. Neil, "Predicting football results using Bayesian nets and other machine learning techniques," *Knowledge-Based Systems*, vol. 19, pp. 544–553, 2006.

[7] V. Kuleshov, N. Fenner, and S. Ermon, "Accurate uncertainties for deep learning using calibrated regression," in *Proc. ICML*, 2018.

[8] B. Loeffelholz, E. Bednar, and K. W. Bauer, "Predicting NBA games using neural networks," *J. Quantitative Analysis in Sports*, vol. 5, no. 1, 2009.

[9] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. ICML*, 2005, pp. 625–632.

[10] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, 1999, pp. 61–74.

[11] N. Silver, "How our NBA predictions work," *FiveThirtyEight*, 2015. [Online]. Available: https://fivethirtyeight.com

[12] N. Tax and Y. Joustra, "Predicting the Dutch football competition using public data: A machine learning approach," *Trans. Knowledge Data Eng.*, vol. 10, no. 10, 2015.

[13] B. Yurdakul, "Statistical properties of population stability index," Working paper, 2018.

[14] A. Zimmermann, "Basketball predictions in the NCAAB and NBA: Similarities and differences," *Statistical Analysis and Data Mining*, vol. 9, no. 5, pp. 350–364, 2016.